

High-Performance Computing and AI Architectures
CESE5040

Lab Assignment 1

Christos Strydis
Rajendra Bishnoi
Amirreza Movahedin

2026

Contents

1	Introduction	2
2	The Virtual Brain (TVB)	2
3	TVB Algorithm	3
4	Sequential Implementation	4
5	Vectorization	6
6	Just-in-time (JIT) compilation	7

Submission guidelines:

- Hand in the lab report in pdf format with the following name:
Lab1.Firstname.Lastname.Studentnumber (for example: Lab1.Pieter.deBoer.123456)
- Hand in the Python scripts (the **.py** files) as separate, unzipped Python scripts. Do not forget to add meaningful comments to your code. Handing in poorly annotated code will be penalized with one point.
- You must submit your code for exercises **1.6**, **1.7**, **1.8**, and **1.9** alongside your report for this lab. Please name your script files with the following convention: L1-Qx where x is the exercise number.

1 Introduction

The goal of the first three lab assignments is to accelerate the simulation of a prominent brain model called *The Virtual Brain (TVB)*. You will be doing the acceleration using the techniques discussed in the lectures. In this assignment, you will (1) **analyze** the TVB algorithm in terms of its computation needs, (2) **profile** the sequential version of the code to evaluate its performance, (3) explore **vectorization** opportunities to improve the performance, and (4) use **just-in-time (JIT)** compilation to examine its effect on the execution speed of the algorithm.

2 The Virtual Brain (TVB)

For centuries, understanding the brain and the way it works has been of great concern to scientists. The National Academy of Engineering of the United States has classified reverse engineering the brain as one of the grand challenges of the 21st century [3]. Due to the tremendous leaps in computing capabilities of late, brain simulation has become increasingly important in neuroscientific research. Brain simulation can be performed at different levels of abstraction for different purposes [4]. Nowadays, coarse-grained brain models are developed with the goal of clinical use in mind.

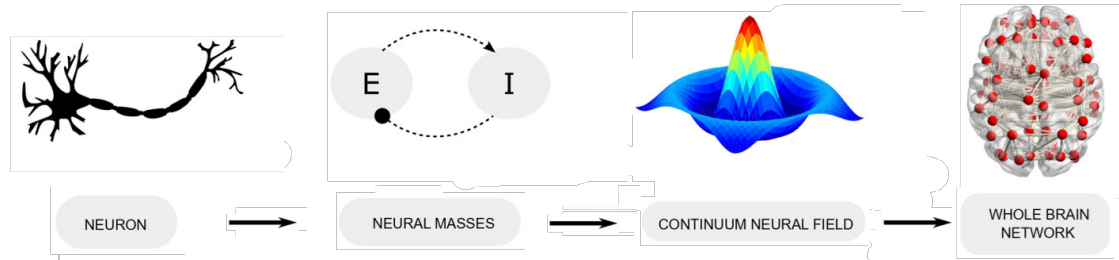


Figure 1: Different abstraction levels of brain modeling [4]

The Virtual Brain (TVB) is one of the leading tools in building and simulating large-scale brain models [5]. By reducing the complexity at the microscopic level, TVB can deliver an overview of the brain's macro-organization. With this modeling strategy, TVB can produce sufficiently accurate brain signals that can be useful in various clinical applications. TVB divides the entire brain into different regions known as *centers*. Each center models the activity of all the neurons in a given brain region, called a *neural mass*. These centers are typically *sparsely* interconnected, affecting each other's activity in a process called *coupling*. The couplings among centers bear different strengths (represented with a weight value) in addition to delays which are due to the limited speed of signal propagation in the brain.

To make the model more accurate for each patient, TVB encodes personal, brain-imaging data in the described base model. This means that the brain region that each center represents, in addition to the weights and delays associated with the centers' connectivity, are patient-specific. For each patient, the personal brain model must be tuned in a process called *model fitting*, shown in Figure 2. This model fitting often requires a large number of simulations for each patient. These fitting strategies are used to infer certain model parameters that are of interest within the setting in which TVB is being used [7]. Since such model-fitting strategies are typically very time-consuming, achieving high-performance execution of TVB models could greatly accelerate this process.

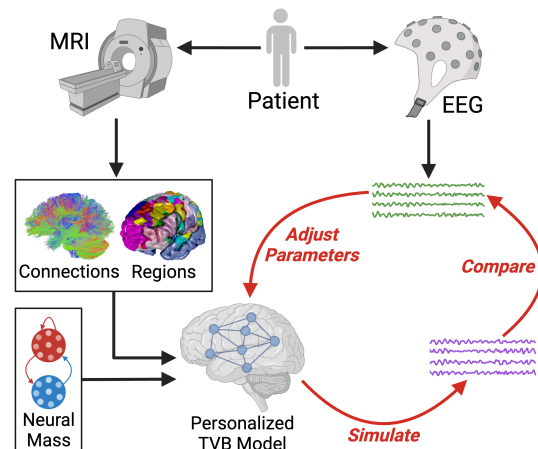


Figure 2: TVB model fitting pipeline [2]. A high-performance TVB simulation can reduce the fitting process time by speeding up the loop marked with red.

3 TVB Algorithm

TVB divides the brain into different regions called centers. These centers are interconnected via different patterns, with each connectivity (or coupling) having a certain strength (or weight) and delay associated with it. Each center contains internal neural activity called *local dynamics*, which comprise the neural-mass model of that center. In addition to local dynamics, each center is also affected by other centers which is referred to as the *coupling input* to the center. If we assume there are N centers in the model, the activity of each center $i \in [1, N]$ over time is described using a differential equation shown in (1).

$$\frac{d\vec{X}_i(t)}{dt} = F(\vec{X}_i(t)) + C_i(t) + u(t) + \eta(t) \quad (1)$$

where $\vec{X}_i \in \mathbb{R}^M$ are the state variables of each center. The activity output of each center is one of these state variables. Function $F: \mathbb{R}^M \rightarrow \mathbb{R}^M$ describes the local dynamics of each center, while $u(t)$ and $\eta(t)$ are the external input to the center and the noise in the system, respectively. Although important for the model's accuracy, we do not consider them further in our lab assignments.

When we talk about simulating the brain model, we mean solving differential equations (1) for all the centers in the model over a certain amount of timesteps. There are many methods to solve differential equations numerically with different accuracy levels and computational costs. The method we will use in this lab is the *Forward Euler (FE)* which provides the required accuracy with minimal computational cost [1, 6].

Equation (2) formulates the computation that must be performed for each center $i \in [1, N]$ of the model at each timestep of the simulation. h is the timestep size of the FE method and t is the timestep of the simulation.

The coupling inputs to each center are represented by $C_i(t)$, which is added to one of the state variables of the center. The $C_i(t)$ for each center $i \in [1, N]$ is calculated as formulated in (3).

$$\forall i \in [1, N]: \vec{X}_i(t+1) = \vec{X}_i(t) + h \cdot \left(F(\vec{X}_i(t)) + C_i(t) \right) \quad (2)$$

$$C_i(t) = K_{post} \left(\sum_{j=1}^N W_{ij} K_{pre} (X_j(t - d_{ij}), X_i(t)) \right) \quad (3)$$

In (3), W_{ij} and d_{ij} represent the weight and delay associated with the connection from center j to center i respectively. Functions $K_{pre} \in (\mathbb{R}, \mathbb{R}) \rightarrow \mathbb{R}$ and $K_{post} \in \mathbb{R} \rightarrow \mathbb{R}$ are pre- and post-synapse functions respectively which scale the signals from other centers to more realistically represent their strength when reaching the receiving center [6]. It is worth noting that the connections between the centers in the model are usually sparse, meaning that a large number of weight values (W_{ij}) in the model are zero.

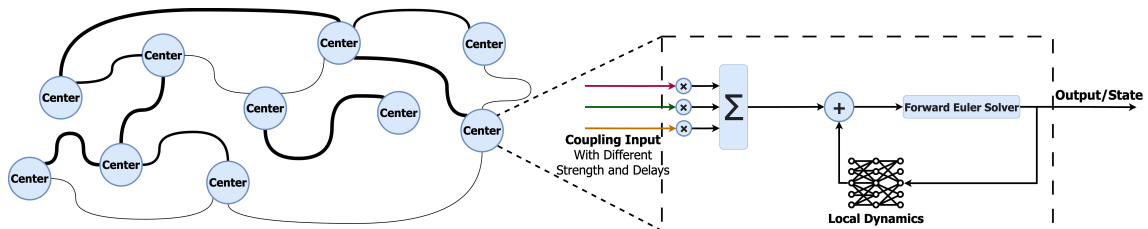


Figure 3: Illustration of the TVB algorithm. **(Left)** The interconnected network of centers with different strengths and distances, which results in different delays. **(Right)** The coupling calculation, local dynamics function, and Forward Euler solver calculations within each center.

4 Sequential Implementation

The sequential implementation of The Virtual Brain is provided in the `tvb_seq.py` script. Run the script to get an idea what it does and how it works. The script consists of mainly 4 parts:

- `simulate(...)`
- `calculate_coupling(...)`
- `step(...)`
- Initialization (if `__name__ == "__main__":`)

The `simulate` function performs the actual TVB simulation by taking in the parameters of it. The parameters include (in order) the weight matrix (`W`), the distance matrix (`D`), the number of centers (`N`), the number of state variables per center (`M`), the timestep size (`dt`), the duration of the simulation (`tf`), the signal propagation speed in the brain (`speed`), and the frequency parameter of the local dynamics (`freq`). In its main body, the `simulate` function loops over all the timesteps of the simulation and calculate the new value of each center's output. It does this by calling `calculate_coupling` and `step` functions.

The `calculate_coupling` function, as its name suggests, calculate the coupling input of a certain center at a certain timestep denoted by `n` and `t` in its argument list, respectively. Additionally, it takes as argument the history of the centers' values (`Xs`) and the related rows in weight (`W.row`) and delay (`D.row`) matrices.

The `step` function performs the Forward Euler integration stepping for a certain center at a certain timestep. It evaluates the local dynamics (by calling function `f`) and performs the Forward Euler solving afterwards. In the end, it returns the new state variable value of the center.

The script's initialization part reads the weight and distance matrices and sets the parameters of the simulation. It then calls the `simulate` function and plots the result. There are 3 standard TVB datasets available with 76, 192, and 998 centers.

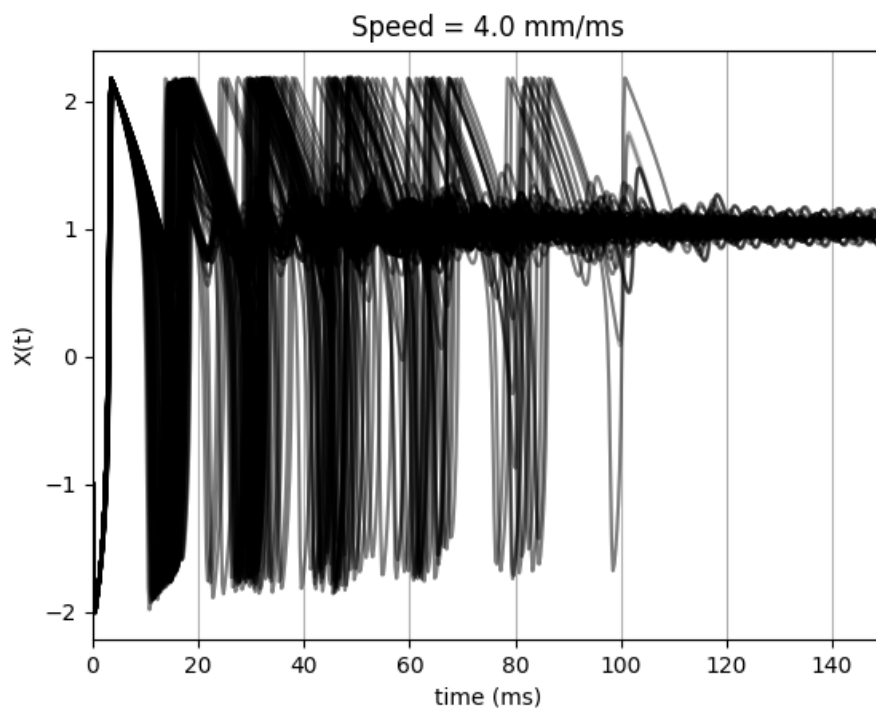


Figure 4: Simulation result for TVB192 running for 150ms

Exercise 1.1 (20 points)

Let's do a pen-and-paper analysis of the TVB algorithm shown in Equations (2) and (3), and coded in the `tvb_seq.py` script.

1. What is the time complexity of the algorithm with regard to the number of centers and total simulation time, respectively? (5 points)
2. Roughly how much memory capacity would be needed (in number of elements) to store state variables for this algorithm? How does it scale *individually* with each of the different simulation parameters N , M , and D ? (5 points)
3. Imagine in Equation (3) the pre-synapse function is linear, meaning it has the form if $K_{pre}(X_j, X_i) = aX_j + bX_i$. Assuming addition and multiplication have the same computational cost, what transformation can you do to Equation (3) to make it more computationally efficient? (5 points)
4. Is it possible to parallelize across timesteps? In other words, can we run the computations of multiple timesteps at the same time? Why (not)? (5 points)

Exercise 1.2 (7.5 points)

Open the `tvb_seq.py` script and instrument the code using `tic-toc` pairs to measure the execution time of the entire simulation and the `coupling_calculation` function. Run the script for **150 timesteps**, and report the time it takes to perform the simulation and the coupling-calculation portion for all 3 different TVB datasets (76, 192, 998 centers). Make sure to place `tic-toc`'s properly, so as to measure the actual computation, excluding any initialization or plotting code. Do the results scale as you predicted in Exercise 1.1? What portion of the entire execution time is spent on `coupling_calculation`? Why do you think that is the case?

Exercise 1.3 (7.5 points)

Override the distance matrix with a matrix that is all zero. This means that there are no delays when centers are communicating with each other. Repeat Exercise 1.2 and report the numbers. Are there any difference in execution times compared to Exercise 1.2? Why is that?

Exercise 1.4 (20 points)

As mentioned in Section 3, the weight matrix (W) in TVB models are most usually sparse, meaning that most of its elements are 0. In this case, we can represent the sparse matrix without storing the zero-valued elements, and hence not performing any computation using them. One of the most common methods is *Compressed Sparse Row (CSR)*. The `tvb_seq_sparse.py` script implements TVB algorithm using sparse computation.

1. If matrix W has an sparsity of $S\%$, which part of the TVB algorithm can potentially be sped up and by approximately how much? (5 points)
2. Open the `tvb_seq_sparse.py` script. The `simulate_sparse` function performs some pre-processing to create the CSR format of the W and D matrices. Examine this part and the `calculate_coupling_sparse` function and understand how they work. What is the sparsity of the TVB192 dataset? (What portion of the W matrix in TVB192 dataset is zero-valued?) (5 points)
3. Run the `tvb_seq_sparse.py` script for TVB192 for **150 timesteps** and report the execution times for the entire simulation and the `calculate_coupling_sparse` function. How does the speed up of the coupling calculation part compare to the theoretical value (using what you analyzed in question 1)? If there are differences, what can explain them? (5 points)

4. What downsides can you think of for sparse computations? (5 points)

Exercise 1.5 (10 points)

In some other implementations of TVB, the local dynamics function (F) is replaced with a *neural network*, more specifically a *Multi-Layer Perceptron (MLP)*. We will talk about neural networks more, later in the course. For now, you can think about MLPs as basically a series of matrix multiplications. The reason for using MLPs in the place of local dynamics is that MLPs are universal function approximators. This means that they can approximate any local dynamics function just by changing their weight parameters, providing great amount of generality. Additionally, having MLPs means that the dynamics of each center can also be learned to make the model more accurate for each patient. However, MLPs have the downside of being more computationally intensive than explicitly coded local dynamics functions.

1. Open the `tvb_seq_mlp.py` script. The local dynamics function is replaced with an MLP. Run the simulation for TVB192 dataset for **15 timesteps** and report the execution time for the entire simulation, the `step` function, and the `calculate_coupling` function (in non-sparse mode).
2. What portion of the entire simulation is spent on coupling calculation part? What is the maximum theoretical speed up we can achieve by accelerating this part? (5 points)
3. Run the script again, this time with coupling calculation performed in sparse mode. You can do this by switching the `USE_SPARSE` flag in the initialization part to `True`. What is the speed up of the coupling calculation part? What is the speed up of the entire simulation? Does these numbers match what you would get from Amdahl's law? (5 points)

5 Vectorization

In this section, we are going to work with the first parallelization technique, vectorization. Many programs use loops to perform operations on vectors of data, where a vector is an array of elements. You can find many of these loops in the sequential implementation of the TVB algorithm. When a processor executes the instructions in such a loop, the operations are performed one at a time on individual data elements. As a result, many processor instructions need to be executed to process all elements. Modern computer processors, however, can operate on multiple elements in parallel using a single *vector* instruction. This is exactly what vectorization is: It is the process of performing the same computation over a whole vector of data *at a single time*. One important condition for vectorization (and parallelism in general) is that the computations performed in parallel are **independent**. That is, executing the one computation does not affect or is affected by the other computation and, thus, they can take place in parallel.

In Python, the NumPy package is best-suited for vectorization. Among other features, it enables elegant element-wise operations among vector variables. When you perform an operation (for example, multiplication) on an `np.array`, the NumPy package will automatically perform this computation in parallel and in the most efficient way possible.

Exercise 1.6 (12.5 points)

Implement the vectorized version of TVB with no sparsity and MLP as local dynamics (the non-sparse mode of the `tvb_seq_mlp.py` script). Here are some hints to help you vectorize the code.

- Only vectorizing the MLP and coupling calculation functions (f and `calculate_coupling`). You need to change some other parts of the code to make the entire script work. (Optionally, you can think about vectorizing the `for n in range(N)` loop of the `simulate` function as well)
- Replace the lists and nested lists with NumPy arrays.

- One-by-one, starting from the most inner loops, think about how (and whether) you can replace that loop with vectorized operations.
- For the `calculate_coupling` function, take a look at the `np.where()` function for conditioning.
- The local dynamics function `f`, which is an MLP, is basically some vector-matrix multiplications back to back (with some non-linearity in the middle of them). You can take a look at the `np.matmul()` or the `np.dot()` functions for more information. Additionally, use the sliced-up parameters (in the form of NumPy arrays) of the MLP provided in the `lib/mlp_params.py` file.

Report the execution time of your vectorized implementation of the TVB algorithm for the 3 provided datasets (76, 192, and 998 centers) for **15 timesteps**. How much speed up for the entire simulation, and for each part of it are you achieving? (You have to run the sequential implementation with MLPs to for TVB76 and TVB998 as well.)

Exercise 1.7 (12.5 points)

With the help of the `tvb_seq_mlp.py` script (in sparse mode) and the vectorized version you implemented in Exercise 1.6, implement the vectorized version of sparse TVB with MLP. Report the execution time of the entire simulation of **15 timesteps** and the coupling calculation part. Do you see any meaningful performance gain by vectorizing the sparse coupling calculation compared to Exercise 1.6? Explain why.

6 Just-in-time (JIT) compilation

Recall that Python is an interpreted language, meaning that the Python interpreter converts your source code line by line to machine code. The conversion of Python code to machine code thus happens while the instructions are being executed (at execution time). In many other languages, for example C or C++, the source code is first compiled into a machine-code program before being executed. The benefit of pre-compilation is a dramatic reduction in execution time.

Some of the functions in the TVB algorithm are computationally heavy; e.g., they contain big loops. Without pre-compilation, the Python interpreter goes over every instruction in a loop over and over, converting the same piece of code to the same piece of machine code over and over again. If the loops become large, the execution time will become longer as well. Being an interpreted language, Python has certain benefits, however speed is not one of them. To (partially) ameliorate this problem, Python adopts a technique known as *just-in-time* compilation: Whenever we run code using this Python compiler, it will convert the Python source code at the very last moment, *just in time* for execution. We, thus, do not pre-compile the full source code (like in C) but compile the source code just before it gets executed. As a result, we can lower our code-execution time to approximately the same as with C code, which combines the best of both language worlds.

This may all sound quite complex, but using JIT compilation in Python is very easy. There are many modules in Python that offer a compiler, but the easiest one to use is **Numba**. **Numba**, further, has support for the **NumPy** library. JIT compilation is included here as an alternative – and straightforward – means of gaining program performance, however, you should remember that it is *not* a code-parallelization technique.

Exercise 1.8 (5 points)

Open the `tvb_seq_jit.py` script. This is the sequential, non-sparse version of TVB with the MLP. The code is refactored from similar one you saw earlier in this lab to make it compatible with the Numba compiler. Most notably, the nested lists are replaced with NumPy arrays which Numba supports very well. Run this code for TVB76 and TVB192 datasets for **15 timesteps** and note down the execution time of the entire simulation. Then, add appropriate libraries and

`@jit` decorators to use the just-in-time compilation. Measure the execution time of the simulation again. Is there a difference? (Beware not to measure the initial JIT compilation time)

Exercise 1.9 (5 points)

Use just-in-time compilation for the vectorized version of the TVB algorithm that you implemented in Exercise 1.6. You might need to change and refactor your code to make it compatible with Numba compiler. You can refer to the previous exercise as a guide to do that. Is the JIT version of the vectorized code faster than the JIT version of the non-vectorized code? Does performing JIT compilation on the vectorized code yields to as much of a speedup as the non-vectorized version? Why?

References

V. K. Jirsa, T. Proix, D. Perdikis, M. M. Woodman, H. Wang, J. Gonzalez-Martinez, C. Bernard, C. Bénar, M. Guye, P. Chauvel, and F. Bartolomei. The virtual epileptic patient: Individualized whole-brain models of epilepsy spread. 145:377–388, 2017.

A. Movahedin. Tvb pipeline. created in biorender, 2024.

National Academy of Engineering. Grand challenges, 2008.

Anagh Pathak, Dipanjan Roy, and Arpan Banerjee. Whole-brain network models: From physics to bedside. 16. Publisher: Frontiers.

Leon Paula, Knock Stuart, Woodman M., Domide Lia, Mersmann Jochen, McIntosh Anthony, and Jirsa Viktor. The virtual brain: a simulator of primate brain network dynamics. *Frontiers in Neuroinformatics*, 7(10), 2013.

Paula Sanz-Leon, Stuart A. Knock, Andreas Spiegler, and Viktor K. Jirsa. Mathematical framework for large-scale brain network modeling in the virtual brain. 111:385–430, 2015.

Huifang E Wang, Paul Triebkorn, Martin Breyton, Borana Dollomaja, Jean-Didier Lemarechal, Spase Petkoski, Pierpaolo Sorrentino, Damien Depannemaecker, Meysam Hashemi, and Viktor K Jirsa. Virtual brain twins: from basic neuroscience to clinical use. *National Science Review*, 11(5):nwae079, 2024.